MEDIA FORMATION
GAGNER VOTRE VIE PROFESSIONNELLE

Intitulé du cours: JavaScript

Chapitre 1 : Introduction au langage JavaScript

Enseignante: Abid Arij

Année de formation 2025-2026

Plan

- Objectifs du module
- Qu’est-ce que JavaScript ?
- Implantation des scripts
- Notation de base de JavaScript
- Les Variables
- Les Expressions et opérateurs
- Les Fonctions

Objectifs du module

À la fin du module, l'apprenant doit être capable de :

- Comprendre le rôle et la place de **JavaScript** dans le modèle client-serveur
- Apprendre à écrire et exécuter des programmes JavaScript simples
- Utiliser les **boîtes de dialogue** pour interagir avec l'utilisateur
- Manipuler les **variables, les fonctions et les événements** en JavaScript
- Comprendre et utiliser les **objets** dans JavaScript

<script type="text/javascript">
switch (new Date().getDay()) {
case 6:
text = "Friday";
break;
case 0:
text = "Sunday";
break;
default:
text = "Choose Your Day";
}
</script>
Javascript

01

Définition

Qu’est-ce que JavaScript ?

JavaScript est un **langage de programmation** qui permet de rendre les pages web **dynamiques et interactives**.
C’est un langage **interprété** (pas besoin de compilation, le navigateur comprend directement le code).
Il s’exécute **côté client** (dans le navigateur ou browser), contrairement à des langages comme PHP ou Java (côté serveur).
Aujourd’hui, il est aussi utilisé côté serveur grâce à **Node.js**.

JavaScript

Historique

1995 : Brendan Eich crée JavaScript en seulement 10 jours chez Netscape.
Actuellement, la version du langage est le javascript 1.8
Au départ, il s'appelait **LiveScript**, puis renommé **JavaScript** (pour surfer sur la popularité de Java, mais ce sont deux langages différents).
Aujourd'hui, c'est un des **langages les plus utilisés au monde**.

Pourquoi utiliser JavaScript ?

Avec JavaScript tu peux :

- ✓ Afficher des messages, manipuler du texte et des nombres.
- ✓ Réagir aux actions de l'utilisateur (clics, saisies clavier, survol de la souris).
- ✓ Modifier le contenu d'une page sans la recharger.
- ✓ Créer des jeux, des applications web, et même des applications mobiles ou desktop.
- ✓ Créer des **animations**
- ✓ Manipuler le contenu d'une page HTML en direct
- ✓ Gérer des **formulaires** (ex : vérifier qu'un email est valide)

- Voici quelques exemples (on pourrait en citer beaucoup d'autres) de ce que l'on peut en faire dans une page Web :
  - ouvrir des *pop-up* (les petites fenêtres qui s'ouvrent de manière intempestive)
  - faire défiler un texte
  - insérer un menu dynamique (qui se développe au passage de la souris)
  - proposer un diaporama (changement d'image toute les X secondes, boutons pour mettre en pause, aller à l'image précédente / suivante, etc.)
  - avoir une horloge "à aiguilles" (avec la trotteuse)
  - faire en sorte que des images suivent le pointeur de la souris
  - créer de petits jeux
  - insérer des balises du zCode (les balises qui apparaissent en cliquant sur le bouton)

- **Sans JavaScript :**

Un site ne serait qu'une page **statique** (juste du texte, des images, du HTML et du CSS).

- **Avec JavaScript :**

Tu peux ajouter des **animations**,
réagir aux actions de l'utilisateur (clic, saisie clavier, etc.),
modifier le contenu d'une page sans la recharger,
communiquer avec un serveur (ex. récupérer des données d'une API).
=> **HTML** construit la structure, **CSS** habille avec le style, et **JavaScript** donne la vie au site web.

HTML = le squelette d'une maison
CSS = la décoration de la maison
JavaScript = l'électricité qui fait fonctionner les interrupteurs (interactions).

HTML

CSS

JavaScript

la différence entre java et javascript

Critère	Java	JavaScript
Création	1995 par Sun Microsystems (racheté par Oracle)	1995 par Netscape
Type de langage	Orienté objet, langage compilé	Orienté objet mais surtout interprété
Où ça s'exécute ?	Sur la **JVM** (Java Virtual Machine)	Dans le **navigateur** (Chrome, Firefox…) ou sur serveur (Node.js)
Utilisation principale	Applications desktop, Android, back-end (Spring, etc.)	Pages web interactives, animations, back-end avec Node.js
Syntaxe	Très stricte (fort typage, classes obligatoires)	Plus souple, plus simple pour débuter
Compilation	Besoin de compiler (javac MonProgramme.java)	Pas besoin → directement interprété par le navigateur
Exemples d'utilisation	- Applications Android - Applications bancaires - Logiciels lourds (IDE, jeux)	- Boutons interactifs - Formulaires dynamiques - Sites modernes (React, Angular, Vue.js)

02

Implantation des scripts

Déclaration du code JavaScript

- On peut placer du code JavaScript dans une page HTML selon des façons et des formes bien différentes.
    1. **Entre des balises <script> et </script>**
    2. **Code associé à un fichier d'extension .js**
    3. **Code associé à une balise HTML**

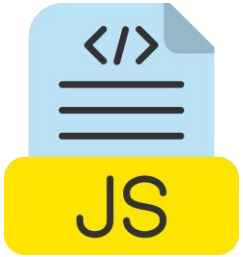
Déclaration du code JavaScript

- On peut placer du code JavaScript dans une page HTML selon des façons et des formes bien différentes.

1. **Entre des balises <script> et </script>**

- Quand ces balises sont placées à l'intérieur du <body> le code JavaScript sera directement interprété au chargement de la page.
- Quand ces balises sont placées dans le <head> le code ne sera pas interprété directement. Il le sera lorsque on y référence dans le <body>. Généralement, on définit des fonctions JavaScript dans le <head> qui seront appelées dans le <body>.

Déclaration du code JavaScript



2. Code associé à un fichier d'extension .js

- On peut séparer le code JavaScript dans un fichier ayant l'extension .js puis l'insérer dans la page web.
- Syntaxe : `<script src="fichier.js"></script>`
- Cela permet de réutiliser le code JavaScript dans plusieurs pages web.
- Ces fichiers contiennent, généralement, la définition des fonctions JavaScript qui seront appelées dans le body des pages web

Déclaration du code JavaScript

3. Code associé à une balise HTML

- Le code JavaScript peut s'insérer sous forme d'un appel de fonction, affectée à un gestionnaire d'événement qui apparaît comme un attribut dans une balise .
- Syntaxe : <balise onevenement="fonction JS" >
- Exemple :

```
<html>
    <head>
        <title>Cours JavaScript</title>
        <meta charset="utf-8">

    </head>

    <body>
        <h1>Titre principal</h1>
        <button onclick="alert('Bonjour !')">Cliquez ici</button>

    </body>
</html>
```

```
<html>
 <head>
  <title> Titre de la page Web</title>
 </head>
 <body>
 <script type="text/javascript">
 <!--
script
 // -->
 </script>
 </body>
</html>
```

Masquer le script aux navigateurs non compatibles avec JavaScript

Les commentaires

Les commentaires en JavaScript sont semblables à ceux du langage C ou php. En effet on utilise les **doubles slash (//)** pour un commentaire de fin de ligne. Tandis que **/* et */** délimitent les commentaires sur plusieurs lignes.

Exemple:

```
<script language="javascript">
//      Ceci est un commentaire sur une ligne
/*
        Ceci est un commentaire
        sur plusieurs lignes
*/
</script>
```

03

Notation de base de JavaScript

Notation de base de JavaScript

- ✓ Boîtes de dialogue
- ✓ Les variables
- ✓ Les opérateurs
- ✓ Les structures de contrôle

03

Les boîtes de dialogue

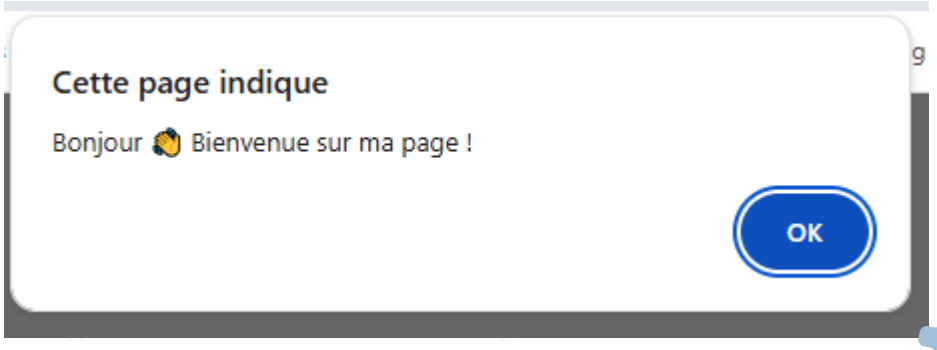
Les boites de dialogues

JavaScript

- Une boîte de dialogue est une fenêtre qui s'affiche à l'écran, et qui permet :
  - Soit d'avertir l'utilisateur (alert)
  - Soit de confirmer un choix (confirm)
  - Soit de lui demander de saisir une information (prompt)

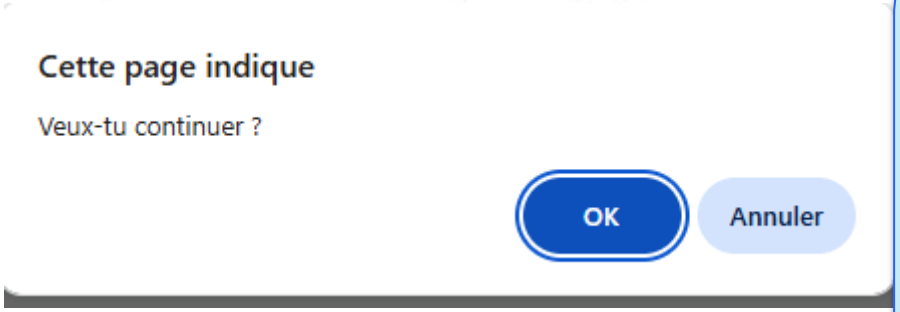
Les boites de dialogues

- **La méthode alert()**
- permet d'afficher une boîte composée d'une fenêtre d'un bouton OK et le texte qu'on lui fournit en paramètre.
- Dès que cette boîte est affichée, l'utilisateur n'a d'autre alternative que de cliquer sur le bouton OK.
- Syntaxe : alert("Attention!");



Les boites de dialogues

- **La méthode confirm()**
- permet à l'utilisateur de choisir entre les boutons "OK" et "Annuler".
- Lorsque l'utilisateur appuie sur "**OK**" la méthode renvoie la valeur **true**. Elle renvoie **false** dans le cas contraire.
- Syntaxe : var c = confirm("message");




```
<script>
  let reponse = confirm("Veux-tu continuer ?");

  if (reponse) {
    alert("Tu as cliqué sur OK ");
  } else {
    alert("Tu as cliqué sur Annuler ");
  }
</script>
```

Les boites de dialogues

- La méthode **prompt()** fournit un moyen simple de récupérer une information provenant de l'utilisateur. La méthode prompt() requiert deux arguments :
  - le texte d'invite
  - la chaîne de caractères par défaut dans le champ de saisie
- Syntaxe : rep = prompt('Question','valeur par défaut');

```
<script>
  let age = prompt("Quel âge as-tu ?");


  if (age >= 18) {
    alert("Tu es majeur ✅");
  } else {
    alert("Tu es mineur ❌");
  }
</script>
```

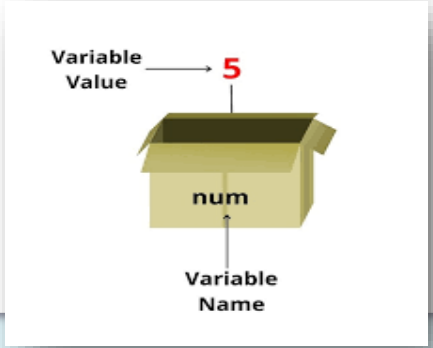
04

Les Variables

Notion de variable:

Une **variable** est un **conteneur** qui permet de **stocker une valeur** pour pouvoir l'utiliser plus tard dans le programme.

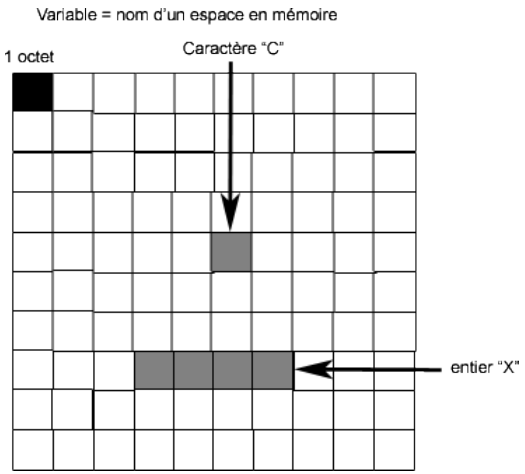
Exemple concret :
Pense à une **boîte** où tu peux mettre un objet (un nombre, un mot, etc.).
La boîte a un **nom** (nom de la variable) pour que tu puisses retrouver son contenu.



Déclaration d'une variable

En JavaScript, on peut déclarer une variable de **3 façons** :

a) **Var**
b) **Let**
c) **const**



a) var

Déclaration **ancienne**, encore utilisée mais moins recommandée.
Exemple :

```
var nom = "Ahmed";
console.log(nom); // Affiche "Ahmed"
```

b) let

Déclaration **moderne**.
La valeur peut changer.
Exemple :

```
<script>
  let prenom = "Arij"; // déclaration de la variable
  alert("Bonjour " + prenom + " !");
 </script>
```

c) const

$\pi = 3.14$

Déclaration d'une **constante**.
La valeur **ne peut pas changer** après l'affectation.
Exemple :

```
const pi = 3.14;
console.log(pi); // 3.14
// pi = 3.1415; → ERREUR !
```

la différence entre var et let en JavaScript,

Critère	var	let
Déclaration	var est l'ancienne façon de déclarer une variable en JavaScript.	let est la nouvelle façon introduite avec **ES6 (2015)**.
Portée (scope)	var a une **portée de fonction**. Cela signifie que la variable est accessible dans toute la fonction.	let a une **portée de bloc**. La variable existe seulement dans le bloc { } où elle est déclarée.
Redéclaration	On peut **déclarer plusieurs fois la même variable** avec var sans erreur.	On **ne peut pas redéclarer** une variable avec let dans le même bloc.
Modification de valeur	La valeur peut être **modifiée**.	La valeur peut aussi être **modifiée**.
Hoisting (remontée)	var est **hoistée** : la variable est créée au début du programme avec la valeur undefined.	let est aussi hoistée mais **ne peut pas être utilisée avant sa déclaration**.
Utilisation recommandée	Peu recommandée aujourd'hui dans les nouveaux programmes.	**Recommandée dans le JavaScript moderne.**

Identification du type d'une variable en JavaScript

En JavaScript, chaque variable a un **type**, par exemple :

- number → nombres
- string → texte
- boolean → vrai/faux
- object → objets, tableaux, null
- undefined → variable déclarée mais non initialisée
- function → fonctions

Pour connaître le type d'une variable, on utilise **l'opérateur typeof**.

Types de valeurs qu'une variable peut contenir

Type	Exemple	Description
String	"Bonjour"	Texte entre guillemets
Number	25, 3.14	Nombre entier ou décimal
Boolean	true, false	Vrai ou faux
Null	null	Valeur vide volontairement
Undefined	non assignée	Variable déclarée mais pas de valeur
Object	{nom:"Ahmed", age:20}	Objet avec propriétés
Array	[1,2,3]	Liste de valeurs

Chaque variable en JavaScript a un **type**, qui définit le genre de données qu'elle contient.

Pour identifier le type d'une variable, on utilise l'opérateur **typeof**

Syntaxe de base

typeof variable

Retourne une **chaîne de caractères** indiquant le type de la variable.

```
let nom = "Ahmed";
console.log(typeof nom); // "string"

let age = 20;
console.log(typeof age); // "number"

let estConnecte = true;
console.log(typeof estConnecte); // "boolean"

let utilisateur = {prenom: "Ahmed", age: 20};
console.log(typeof utilisateur); // "object"
```

```
let x;
console.log(typeof x); // "undefined"

let f = function() { return 0; };
console.log(typeof f); // "function"

let rien = null;
console.log(typeof rien); // "object" (particularité de JS)
```

Bonnes pratiques pour nommer une variable

- Le nom doit **commencer par une lettre, $ ou _**.
- Utiliser **camelCase** pour les noms composés : prenomEtNom.
- Éviter les noms réservés (let, for, if, etc.).
- Pas d'espaces, pas de caractères spéciaux.
- Respecter la **casse** : nom ≠ Nom
- Utiliser des noms **significatifs** pour comprendre ce qu'ils contiennent.

Exemple 1 : Afficher le prénom dans la console

```
let prenom = "Arij";
console.log("Bonjour " + prenom );
```

Choix des identificateurs

Le choix des noms de variables est soumis à quelques **règles** qui varient selon le langage, mais en général :

- Un nom doit commencer par **une lettre alphabétique.**

Exemple **valide**

```
let nom;
let ageUtilisateur;
const PI = 3.14;
let $compteur;
let _score;
```

.

exemple invalide : 1A

```
let 1prenom;   // commence par un chiffre
let mon prénom; // espace
let nom@;      // caractère spécial
```

Exercice

1. Déclarer une variable nom et lui assigner votre prénom.
2. Déclarer une variable age et assigner votre âge.

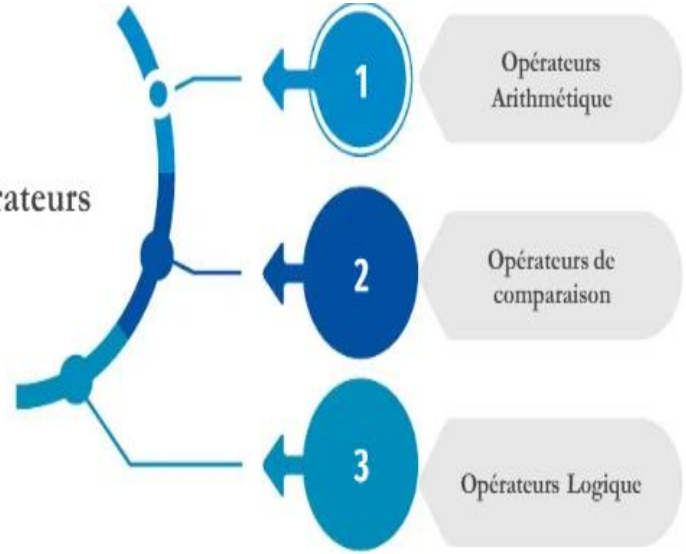
Bonjour, je m'appelle [nom] et j'ai [age] ans.

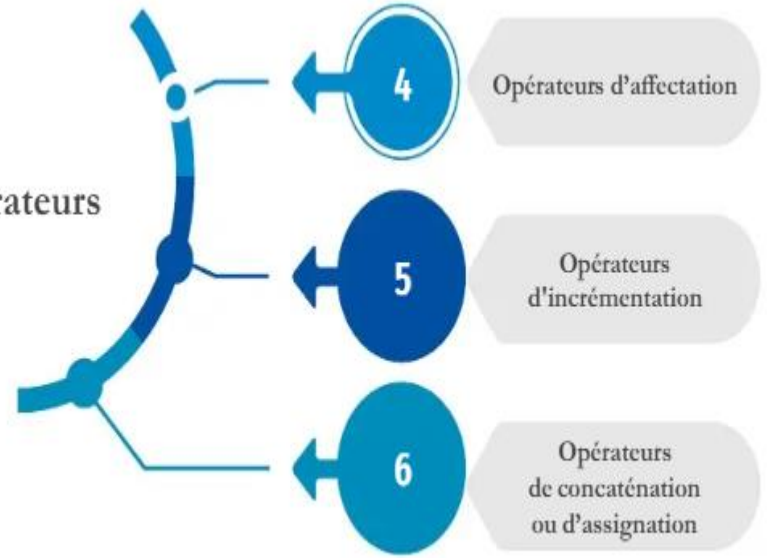
```
// Déclaration des variables
let nom = "Ahmed";
let age = 20;

// Affichage du message
console.log("Bonjour, je m'appelle " + nom + " et j'ai " + age + " ans.");
```

05

Les Expressions et opérateurs

Les opérateurs





Expressions et opérateurs

Une expression est une combinaison de variables, constantes et opérateurs qui produit une valeur.

Les Opérateurs :

- **Arithmétiques :** +, -, *, /, %, DIV
- **comparaison:** == (égal), != (différent), <, >, <=, >
- **Logiques :** ET, OU, NON.

Comparaison	Opérateurs
==	Égal à
<	Inférieur à
>	Supérieur à
<=	Inférieur ou égal à
>=	Supérieur ou égal à
!=	Différent de

✨ a) Opérateurs arithmétiques

Les opérateurs arithmétiques

Opérateur	Dénomination
+	opérateur d'addition
-	opérateur de soustraction
*	opérateur de multiplication
/	opérateur de division
=	opérateur d'affectation
%	opérateur modulo

🔍 b) Opérateurs de comparaison (relationnels)

- Ils comparent deux valeurs et donnent une réponse **vraie ou fausse** (true/false).

Opérateur	Dénomination
== **A ne pas confondre avec le signe d'affectation (=)!!**	opérateur d'égalité
!=	opérateur de différence
<	opérateur d'infériorité stricte
<=	opérateur d'infériorité
>	opérateur de supériorité stricte
>=	opérateur de supériorité

⚙ c) Opérateurs logiques (booléens)

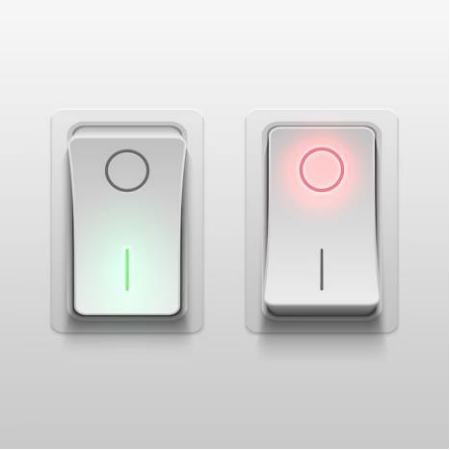
- Ils permettent de **combiner des conditions**.

Opérateur	Signification	Exemple	Résultat
`ET` ou `&&`	Vrai si les deux sont vraies	`5 > 3 ET 2 < 4`	`vrai`
`OU` ou `		`	Vrai si au moins une est vraie
`NON` ou `!`	Inverse le résultat	`NON (5 > 3)`	`faux`

🧠 **exemple:**

Imagine deux interrupteurs :

- **ET** = les deux interrupteurs doivent être ON pour allumer la lumière.
- **OU** = au moins un interrupteur ON suffit.
- **NON** = tu appuies sur un bouton pour faire l'inverse (ON devient OFF).



Les opérateurs associatifs

Opérateur	Exemple	Signification
+=	X+=Y	X=X+Y
-=	X-=Y	X=X-Y
*=	X*=Y	X=X*Y
/=	X/=Y	X=X/Y
%=	X%=Y	X=X%Y

🎯 Priorité des opérateurs – Qui passe en premier ?

Les opérateurs suivent un ordre de priorité :

🏆 **Ordre de priorité (comme les règles de calcul en mathématiques) :**

1. **Parenthèses** ( )
2. **Multiplication** *, **Division** /, **Modulo** %
3. **Addition** +, **Soustraction** -
4. **Comparaisons** ==, !=, >, <, >=, <=
5. **Logiques** : NON → ET → OU

🧠 **Astuce :**
Utilise des **parenthèses** pour forcer l'ordre, comme en mathématiques.

Opérateur	Nom	Compare la valeur ?	Compare le type ?	Exemple
==	Égalité non stricte	Oui	**Non**	"5" == 5 → true
===	Égalité stricte	Oui	Oui	"5" === 5 → false

```
let a = "10";
let b = 10;

console.log(a == b);   // true → car "10" est converti en 10
console.log(a === b);  // false → types différents
```

Conversion des variables en JavaScript

JavaScript est un langage **faiblement typé**, ce qui signifie que **le type d'une variable peut changer automatiquement** selon le contexte, ou peut être converti volontairement par le programmeur.

- **Conversion automatique (implicite)**

JavaScript peut **convertir un type en un autre** automatiquement :

```
let x = "5";  // string
let y = 2;    // number

let resultat = x + y;
console.log(resultat); // "52" → la number 2 est convertie en string
```

Exercice : Calculer la somme de deux nombres

Énoncé :

- Demander à l'utilisateur de saisir **deux nombres** avec prompt().
- Calculer la **somme** de ces deux nombres.
- Afficher le résultat avec alert().

```
let nombre1 = prompt("Entrez le premier nombre :");
let nombre2 = prompt("Entrez le deuxième nombre :");
```

- ⚠ **Attention : prompt() retourne toujours une chaîne de texte, même si l'utilisateur tape un nombre.**

Convertir les valeurs en nombres

Pour effectuer un calcul, il faut transformer les chaînes en nombres.

On peut utiliser parseInt() pour des entiers ou parseFloat() pour des nombres décimaux.

```
<script>
   let nombre1 = prompt("Entrez le premier nombre :");
   let nombre2 = prompt("Entrez le deuxième nombre :");

   // Conversion en nombres
   nombre1 = parseInt(nombre1);
   nombre2 = parseInt(nombre2);

   // Calcul de la somme
   let somme = nombre1 + nombre2;

   // Affichage du résultat
   alert("La somme de " + nombre1 + " et " + nombre2 + " est : " + somme);
 </script>
```

QUIZ!

Question1:

Que fait ce code ?

```
console.log("Bonjour !");
```

- Crée une alerte avec "Bonjour !"
- Écrit "Bonjour !" dans la console
- Demande "Bonjour !" à l’utilisateur

Question 2:

Quelle est la sortie du code suivant ?

```
let x = true;
let y = false;
console.log(x && y);
```

- true
- false
- 1
- Erreur

Question 3:
Que fait ce code ?

```
let age = prompt("Quel est ton âge ?");
```

- Affiche l’âge dans la console
- Demande à l’utilisateur de taper son âge dans une boîte
- Crée une variable sans valeur

Question 5:

Que fait alert("Hello") ?

- Affiche "Hello" dans la console
- Rien du tout
- Ouvre une boîte de dialogue avec "Hello"

Merci pour votre attention